

NEXIS: A Multi-Tenant Agentic Pipeline for Supervised Autonomous Fault Recovery

System Architecture, Algorithms, and Evaluation Harness

RATUL SIKDER, Independent Research / Masters Project, New Zealand

We present NEXIS, a multi-tenant Software-as-a-Service platform that closes the production fault-recovery loop with a fleet of nine cooperating Large Language Model (LLM) agents arranged in two layers (L2 cognitive: *Sentinel*, *Pathfinder*, *Synthesiser*; L1 executive: *Architect*, *Backend*, *QA*, *DevOps*, *DataEngineer*) plus a deterministic *ApprovalGate*. The pipeline ingests webhook events from Sentry, ArgoCD, and OpenTelemetry; performs windowed anomaly detection; queries a code-dependency graph in Neo4j with do-calculus root-cause analysis (DoWhy); generates a candidate patch through retrieval-augmented LLM synthesis; validates the patch inside a network-less Docker sandbox; routes by severity through a human approval gate; and opens a pull request on the tenant's repository via a scoped GitHub App. The control plane is a Go modular monolith; validator and gitops are carved out as separate services because each holds a hard security boundary. Tenant isolation is enforced at the database layer through Postgres Row-Level Security keyed by a SET LOCAL GUC. We describe the system model, present pseudocode for each agent layer, the severity-routed signal/timer race used by the approval gate, the JSON-schema retry policy applied uniformly to L1 agents, and the per-tenant token-budget enforcement. We also describe the dual-provider evaluation harness used to compare gpt-4o against locally hosted qwen2.5-coder:14b on a 30-fault benchmark with five seeds per fault.

Additional Key Words and Phrases: LLM agents, fault recovery, multi-tenant SaaS, workflow orchestration, row-level security, do-calculus, sandbox validation, GitOps

1 Introduction

Modern service operators spend a disproportionate share of engineering capacity on *toil*: triaging alerts, hand-writing post-mortems, and applying repetitive patches under time pressure. Recent advances in instruction-tuned LLMs make it tractable to delegate parts of this loop — diagnosis, patch drafting, test scaffolding — to autonomous agents. However, naively wiring an LLM to a production deployment pipeline is operationally indefensible: bad patches can be auto-deployed, sandbox escape can leak tenant credentials, and unbounded LLM spend can bankrupt a small operator. NEXIS addresses these failure modes simultaneously through (i) a closed-loop, observable, replay-deterministic workflow on Temporal; (ii) strict service-level isolation for the two surfaces that matter (validator sandbox, GitOps credential); (iii) severity-routed human supervision; and (iv) a per-tenant pre-call token budget with hard kill switches.

The system targets three concurrent goals: (a) SaaS productisation (10 design partners by month 6); (b) a Masters thesis whose contribution is the integrated architecture; and (c) an empirical evaluation comparing closed and open-weight LLM stacks under an identical fault-injection harness.

Contributions.

- (1) A modular-monolith control plane with a nine-agent plane and two carved-out security services, justified against the alternative of nine independent microservices (Sec. 2).
- (2) Pseudocode for each algorithmic layer: L2 detection (Algorithm 1), L2 causal RCA (Algorithm 2), L2 planning (Algorithm 3), L1 generic agent with JSON-schema retry (Algorithm 4), the ApprovalGate severity router with deterministic signal/timer race (Algorithm 5), the orchestrating recovery DAG (Algorithm 6), and the per-tenant pre-call token budget (Algorithm 7).

- (3) A database-level multi-tenancy proof obligation discharged by Postgres Row-Level Security and an integration-test fuzzer (Sec. 2.2).
- (4) A dual-provider evaluation harness comparing gpt-4o and qwen2.5-coder:14b on the same incident, with token, cost, latency, and patch-correctness instrumentation (Sec. 5).

2 System Architecture

NEXIS is deployed as four backend processes and one frontend:

- (1) control-plane (Go 1.22 + Chi): HTTP API, Temporal worker, 9-agent plane.
- (2) validator (Go): receives (*repo_sha*, *patch_diff*) and runs the patched image inside docker run network=none read-only tmpfs /tmp. Property tests via Hypothesis.
- (3) gitops (Go): mints short-lived GitHub App JWTs (10 min TTL) and opens pull requests with the patch, agent transcript, and risk score. Sole holder of the GitHub App private key.
- (4) causal-inference (Python, gRPC :8090): DoWhy do-calculus over metric series for backdoor-adjusted root-cause hypothesis.
- (5) apps/web (Next.js 16.2.2): landing and console; subscribes to a Server-Sent Events stream for the live pipeline timeline.

The pipeline runs on Temporal so that workflow history is replay-deterministic; activities outside the workflow (such as approval signals) are delivered through signal channels that are themselves replay-safe.

2.1 Why a Modular Monolith, Not Nine Microservices

The nine agents are logical roles, not deployment units. A microservice per agent would multiply the number of IAM roles, dashboards, contract artefacts, and distributed traces by 9, in exchange for no runtime benefit a solo developer can capture in the launch window. We keep the agent plane inside one binary with strict package boundaries enforced by go-arch-lint. Two services are split out where the separation buys a security property:

- **Validator** is isolated because untrusted code is executed there. The blast radius of a sandbox escape must not include the control plane's database connection or the GitHub App key.
- **GitOps** is isolated because it is the *only* process with permission to read the GitHub App private key. A compromise of the control plane that did not also compromise GitOps cannot mint a JWT.

2.2 Multi-Tenancy via Row-Level Security

Every tenant table carries an *org_id* column and enables Postgres RLS in FORCE mode:

```
ALTER TABLE incidents_raw ENABLE ROW LEVEL SECURITY;
ALTER TABLE incidents_raw FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON incidents_raw
  USING (org_id = current_setting('app.current_org_id')::uuid)
  WITH CHECK (org_id = current_setting('app.current_org_id')::uuid);
```

The application connects as role *nexis_app*, which is not *BYPASSRLS*. Every transaction issued by the application calls `set_config('app.current_org_id', $1, true)` as its first statement; `true` scopes the GUC to the transaction, eliminating leakage across pooled connections. The migration role *nexis* is the only role with *BYPASSRLS* and is used only by *golang-migrate*. An integration-test fuzzer creates rows under *org_a* and *org_b*, sets the GUC to *org_a*, and

asserts that `SELECT *` returns zero rows belonging to `org_b`. This test is run for every tenant table on every CI build and constitutes the system’s SOC 2 cross-tenant evidence.

3 Algorithms

We now describe the algorithmic content of each layer. We use the notation $\mathcal{O}$ for the calling organisation, ρ for the agent role, W for a Temporal workflow run, and Σ for a JSON Schema.

3.1 L2 Sentinel: Windowed Anomaly Detection

Sentinel ingests Sentry, ArgoCD, and OpenTelemetry webhook events into `incidents_raw`. A Temporal cron polls a sliding window of size $\tau_w = 60$ s and applies a small rule set: *burst*, *level_shift*, and *novelty*. The *novelty* rule embeds the event title via the tenant’s preferred embedding model and asks pgvector for the nearest neighbour; if the cosine distance to all prior events exceeds a threshold δ_n (default 0.35), the event is treated as novel and forwarded for diagnosis. Severity is initialised from the source level and refined by the rule that fires (Algorithm 1).

Algorithm 1 L2 Sentinel.Detect – windowed anomaly aggregation

Input: organisation $\mathcal{O}$, window τ_w , thresholds θ_{burst} , θ_{shift} , δ_n

Output: incident decision $I = \{\text{detected}, \text{severity_hint}, \text{evidence}\}$

```

1:  $W \leftarrow \text{QUERY}(\text{incidents\_raw where org\_id} = \mathcal{O} \wedge \text{received\_at} \geq \text{now} - \tau_w)$ 
2: if  $|W| = 0$  then
3:   return  $\{\text{detected: false}\}$ 
4: end if
5:  $g \leftarrow \text{GROUPBY}(W, \text{service}, \text{level})$ 
6:  $R \leftarrow \emptyset$ 
7: for all  $(s, \ell, n) \in g$  do
8:   if  $n \geq \theta_{burst}$  then
9:      $R \leftarrow R \cup \{(\text{burst}, s, \ell, n)\}$ 
10:  end if
11: end for
12:  $\mu, \sigma \leftarrow \text{HISTORICALRATE}(\mathcal{O}, s, \ell, 24\text{h})$ 
13: if  $n > \mu + 3\sigma$  then
14:    $R \leftarrow R \cup \{(\text{level\_shift}, s, \ell, n)\}$ 
15: end if
16:  $e \leftarrow \text{EMBED}(\text{title}(W_{\text{latest}}))$ 
17:  $d \leftarrow \text{PGVECTORNEAREST}(\mathcal{O}, e)$ 
18: if  $d > \delta_n$  then
19:    $R \leftarrow R \cup \{(\text{novelty}, d)\}$ 
20: end if
21: if  $R = \emptyset$  then
22:   return  $\{\text{detected: false}\}$ 
23: end if
24:  $\text{sev} \leftarrow \text{SEVERITYFROMRULES}(R)$ 
25: return  $\{\text{detected: true}, \text{severity\_hint: sev}, \text{evidence: } R\}$ 

```

▷ *novelty* → *high*; *burst* → *medium*

3.2 L2 Pathfinder: Causal Root-Cause Analysis

Pathfinder consults the tenant’s code-dependency graph (Neo4j) for upstream candidates and then calls the Python causal sidecar to compute a do-calculus backdoor adjustment of $\Pr[\text{symptom} \mid \text{do}(\text{candidate})]$ over a metric history [1]. The hypothesis with the highest adjusted effect that also has a non-trivial graph path is returned (Algorithm 2).

Algorithm 2 L2 Pathfinder.Diagnose – causal RCA on codegraph

Input: detected incident I , organisation O

Output: root-cause report $\mathcal{R}$ with ranked candidates

```

1:  $s \leftarrow I.\text{service}$ 
2:  $C \leftarrow \text{NEO4JCYPHER}(\text{MATCH}(s) \leftarrow [*1..k](c) \text{ RETURN } c)$  ▷  $k = 3$ 
3: if  $C = \emptyset$  then
4:   return {candidates:  $\emptyset$ , fallback: heuristic}
5: end if
6:  $M \leftarrow \text{LOADMETRICSERIES}(O, s \cup C, 24\text{h})$ 
7:  $\hat{\mathcal{T}} \leftarrow \emptyset$ 
8: for all  $c \in C$  do
9:    $\tau_c \leftarrow \text{CAUSALSIDECAR.ESTIMATE}(M, \text{treatment} = c, \text{outcome} = s)$ 
10:   $\hat{\mathcal{T}} \leftarrow \hat{\mathcal{T}} \cup \{(c, \tau_c.\text{ate}, \tau_c.\text{p\_value})\}$ 
11: end for
12: sort  $\hat{\mathcal{T}}$  by ATE descending, filter  $p < 0.05$ 
13:  $\mathcal{R} \leftarrow \text{top-3 of } \hat{\mathcal{T}} \text{ with graph path summaries}$ 
14:  $\text{PERSIST}(\text{root\_cause\_reports}, \mathcal{R})$ 
15: return  $\mathcal{R}$ 

```

3.3 L2 Synthesiser: Plan and Delegate

The Synthesiser drafts a structured *plan* by retrieving from `code_embeddings` the top- k chunks most relevant to the root-cause hypothesis and instructing an LLM (Algorithm 3). The plan declares which L1 agents should run, in what order, and with which preconditions. The DAG (Algorithm 6) consumes that plan as input but is itself static – the L2 plan informs prompts, not control flow, to keep Temporal history deterministic.

Algorithm 3 L2 Synthesiser.Plan – retrieval-augmented planning

Input: root-cause report $\mathcal{R}$, organisation O , top- k retrieval cap

Output: plan $\mathcal{P} = \{\text{steps}, \text{retrieved_context}\}$

```

1:  $q \leftarrow \text{CONSTRUCTQUERY}(\mathcal{R}.\text{top})$ 
2:  $E \leftarrow \text{EMBED}(q)$ 
3:  $C \leftarrow \text{PGVECTORTOPK}(O, E, k)$  ▷ RLS-scoped
4:  $\pi \leftarrow \text{LOADPROMPT}(\text{synthesiser.plan.v} \geq 1)$ 
5:  $\mathcal{P} \leftarrow \text{LLM.STRUCTURED}(\pi, \mathcal{R}, C, \Sigma_{\text{plan}})$ 
6: return  $\mathcal{P}$ 

```

3.4 L1 Generic Agent Activity

All five L1 agents (Architect, Backend, QA, DevOps, DataEngineer) share one activity shape (Algorithm 4). Three properties are uniform: (i) the LLM is called only through `adapter/llm.Provider`; (ii) the response is parsed against a

JSON Schema Σ_ρ and retried at most $r_{\max} = 2$ times on schema violation; (iii) tokens, model, provider, latency, and cost are written to token_ledger regardless of outcome.

Algorithm 4 L1 generic agent activity ($\rho \in \{\text{Architect, Backend, QA, DevOps, DataEng}\}$)

Input: workflow context W , role ρ , prior outputs Π , schema Σ_ρ , retry cap $r_{\max}$

Output: structured payload y

```

1: CHARGEBUDGET( $O, \rho$ ) ▷ Alg. 7; fails closed
2:  $\pi_\rho \leftarrow \text{LOADPROMPT}(\rho)$ 
3:  $x \leftarrow \text{BUILDINPUT}(\pi_\rho, \Pi)$ 
4:  $r \leftarrow 0, y \leftarrow \perp$ 
5: while  $r \leq r_{\max}$  do
6:    $t_0 \leftarrow \text{now}$ 
7:    $\text{resp} \leftarrow \text{LLM.GENERATE}(\rho, x)$ 
8:    $t \leftarrow \text{now} - t_0$ 
9:    $\text{LEDGERINSERT}(W, \rho, \text{resp.usage}, t)$ 
10:  if  $\text{VALIDATEJSON}(\text{resp.body}, \Sigma_\rho)$  then
11:     $y \leftarrow \text{PARSE}(\text{resp.body})$ 
12:    break
13:  end if
14:   $r \leftarrow r + 1$ 
15:   $x \leftarrow \text{APPENDSCHEMAERRORHINT}(x, \Sigma_\rho)$ 
16: end while
17: if  $y = \perp$  then
18:   return  $\text{FALLBACK}(\rho)$  ▷ stub payload; degrades demo gracefully
19: end if
20: return  $y$ 

```

3.5 ApprovalGate: Signal/Timer Race

The ApprovalGate is deterministic and does not call an LLM. It maps the synthesised patch to a severity in {low, medium, high} using rule-based features (changed-files count, blast-radius from Pathfinder, prior auto-approve success in the last 30 days). Severity drives the signal/timer race shown in Algorithm 5. The race uses Temporal’s Selector, Timer, and signal channels, all of which are replay-safe; mutating workflow state outside selector callbacks would break replay determinism and is forbidden.

3.6 Recovery DAG Orchestration

The workflow body (Algorithm 6) sequences three L2 activities, three L1 activities, runs DevOps and DataEngineer in parallel, joins on the ApprovalGate, and conditionally calls validator and gitops if approved. Every activity emits one activity_events row at *started* and one at terminal status; the workflow itself emits a Pipeline.Complete event last so the Server-Sent Events stream knows to close.

3.7 Per-Tenant Token-Budget Enforcement

Pre-call budget enforcement is the only defence against runaway LLM spend that survives a buggy agent loop. The token_budgets table holds one row per organisation per monthly period with allowed_tokens_in/out and the running used counters. The middleware in Algorithm 7 is consulted before the provider HTTP call; on over-budget it

Algorithm 5 ApprovalGate severity router with signal/timer race

Input: severity σ , signal channel sigCh, medium timeout $\tau_m = 120$ s**Output:** approval signal $\delta \in \{\text{auto_approved}, \text{approved}, \text{rejected}, \text{timeout_rejected}\}$

```

1: if  $\sigma = \text{low}$  then
2:   return  $\{\delta : \text{auto\_approved}, \text{notes} : \text{"low severity"}\}$ 
3: end if
4: NOTIFYSLACK( $W, \sigma$ ), PERSISTDECISION(pending)
5: if  $\sigma = \text{medium}$  then
6:    $T \leftarrow \text{NewTIMER}(\tau_m)$ 
7:    $S \leftarrow \text{NewSELECTOR}$ 
8:    $S.\text{addReceive}(\text{sigCh}, \lambda \text{ msg} \rightarrow \delta \leftarrow \text{msg}; \text{cancel}(T); \text{done})$ 
9:    $S.\text{addFuture}(T, \lambda \rightarrow \text{if not done then } \delta \leftarrow \text{timeout\_rejected})$ 
10:   $S.\text{select}()$ 
11:  return  $\delta$ 
12: else if  $\sigma = \text{high}$  then
13:    $\delta \leftarrow \text{sigCh.receive}()$   $\triangleright$  blocks until human responds
14:   return  $\delta$ 
15: else
16:   error unknown severity  $\triangleright$  fail closed — never auto-approve
17: end if

```

Algorithm 6 RecoveryPipeline workflow body

Input: pipeline input $\mathcal{I} = (O, \text{workspace}, \text{incident})$ **Output:** terminal state of W

```

1:  $\Pi \leftarrow \emptyset$   $\triangleright$  prior outputs accumulator
2: for all  $\rho \in \langle \text{Sentinel}, \text{Pathfinder}, \text{Synthesiser}, \text{Architect}, \text{Backend}, \text{QA} \rangle$  do
3:    $r \leftarrow \text{EXECUTE}(\rho, \Pi)$ 
4:   if  $r.\text{status} = \text{failed}$  then
5:     return failed
6:   end if
7:    $\Pi[\rho] \leftarrow r.\text{structured}$ 
8: end for
9:  $r_{\text{do}}, r_{\text{de}} \leftarrow \text{PARALLELEXECUTE}(\text{DevOps}, \text{DataEng}, \Pi)$ 
10:  $\Pi[\text{DevOps}] \leftarrow r_{\text{do}}, \Pi[\text{DataEng}] \leftarrow r_{\text{de}}$ 
11:  $g \leftarrow \text{EXECUTE}(\text{ApprovalGate.Route}, \Pi)$   $\triangleright$  computes severity
12:  $\delta \leftarrow \text{Algorithm 5}(g.\text{severity})$ 
13:  $\text{EXECUTE}(\text{ApprovalGate.Finalize}, \delta)$ 
14: if  $\delta \in \{\text{rejected}, \text{timeout\_rejected}\}$  then
15:   return failed (non-retryable)
16: end if
17:  $v \leftarrow \text{VALIDATOR}(\text{patch\_diff}, \text{repo\_sha})$ 
18: if  $\neg v.\text{tests\_passed}$  then
19:   return failed
20: end if
21:  $\text{GITOPS.OPENPR}(\text{patch}, \text{transcript}, \text{risk\_score})$ 
22: return succeeded

```

returns `ErrBudgetExceeded` and the activity short-circuits to a stub fallback. Ollama runs are charged at zero dollars but still consume the in-token counter, so we can monitor demand without spending.

Algorithm 7 Pre-call token budget middleware

Input: organisation O , agent ρ , predicted input tokens $\hat{t}_{in}$

Output: permit $p \in \{\text{ok}, \text{deny}\}$

```

1:  $B \leftarrow \text{LOOKUP}(\text{token\_budgets where org} = O \wedge \text{period covers now})$ 
2: if  $B = \perp$  then
3:    $B \leftarrow \text{PROVISION}(\text{default monthly cap})$ 
4: end if
5: if  $B.\text{used\_in} + \hat{t}_{in} > B.\text{allowed\_in}$  then
6:    $\text{AUDIT}(\text{budget\_denied}, O, \rho, \hat{t}_{in})$ 
7:   return deny
8: end if
9:  $\text{INCREMENT}(B, \hat{t}_{in}, 0)$   $\triangleright$  post-call adjusts with actual usage
10: return ok
  
```

4 Security Boundaries

Three boundaries carry the bulk of the security argument.

4.1 Validator Sandbox

The validator container launches a child container with `network=none`, `read-only`, `tmpfs /tmp`, no IAM, and no environment passthrough. The fixture repository is mounted read-only. Any patch behaviour that requires the network (data exfiltration, callback) fails by construction. The interface contract is identical to the Phase 7 Modal.com swap, so the cloud transition is one adapter change.

4.2 GitHub App Credential Boundary

The GitHub App private key is bind-mounted only into the gitops container. Control-plane has no path to read it. The minted JWT has a 10-minute TTL and is scoped to a single installation; the installation token is requested per call rather than cached longer than five minutes. All PR creations are logged to the immutable audit log with the hash chain (see below).

4.3 Audit Hash Chain

`audit_log` rows carry `prev_hash` and `row_hash`. On insert, the row hash is computed as

$$h_i = H(h_{i-1} || \text{org_id} || \text{action} || \text{target} || \text{metadata} || \text{ts})$$

with $H = \text{HMAC-SHA-256}$ under `AUDIT_SECRET`. Tampering with any historical row breaks the chain on the next anchor. A nightly anchor publishes h_n to S3 Object Lock (Phase 7) to make tampering externally falsifiable.

5 Evaluation Harness

The evaluation track is decoupled from the build phases and runs against a synthetic fault-injection harness with 30 fault classes (null-deref, schema-drift, OOM, deadlock, deploy-fail, race, configuration drift, etc.) and 5 seeds per class. Each

(fault, seed) is run twice: once with LLM_PROVIDER=openai (gpt-4o for synthesis, gpt-4o-mini for classifications), once with LLM_PROVIDER=ollama (qwen2.5-coder:14b for code, llama3.1:8b elsewhere). Each run writes one eval_runs parent row and one eval_transcripts row per (provider, agent). Metrics:

- **MTTR**: wall-clock from incident ingest to PR open.
- **Fix-precision**: validator pass rate on first attempt.
- **Fix-recall**: fraction of injected faults where the pipeline closed the loop.
- **\$/incident**: cost per closed loop (Ollama charged at \$0).
- **Human-intervention rate**: fraction of medium-severity runs requiring an explicit signal before the 120 s timer.
- **Schema-validity rate**: fraction of L1 LLM responses passing Σ_p on first try.

For human factors, a within-subjects NASA Task-Load Index study (planned, $n = 12$) compares NEXIS against a PagerDuty-plus-human baseline; the Wilcoxon signed-rank test is the planned non-parametric primary analysis.

6 Threats to Validity

The eval harness uses synthetic faults; production-fault distribution is heavier-tailed and includes faults Pathfinder’s codegraph cannot reach (third-party regressions). The DoWhy adjustment relies on a correctly specified DAG; we mitigate by reporting both raw and adjusted estimates and flagging adjustment-sensitivity. The LLM-provider comparison is confounded by prompt cache state for the OpenAI path; we re-warm the cache before each (fault, seed) and report cache hit rate alongside latency. RLS is verified only for tables we own; webhook-ingested raw payloads can leak tenant context inside JSON columns and are therefore redacted before storage.

7 Related Work

The closest agentic systems are SWE-agent, Devin, OpenHands, and AutoCodeRover for the LLM-agent side, and PagerDuty, FireHydrant, and Rootly for the incident-response side; NEXIS is positioned as the join: an agentic LLM stack with severity-routed human supervision, sandbox-validated patches, and a deployment exit through GitOps. The causal RCA layer is most closely related to do-calculus pipelines [1] adapted to time-series telemetry.

8 Conclusion

NEXIS demonstrates that a single Go binary can host nine cooperating LLM agents behind a Temporal workflow, gate them through a deterministic severity router, validate their output inside a network-less sandbox, and ship the result through a GitHub App, while honouring row-level multi-tenancy and a per-tenant token budget. The architecture trades the optionality of nine microservices for the shippability of a modular monolith with two carved-out security services, and the algorithmic pieces — detection, causal RCA, planning, JSON-schema-retrieved L1 synthesis, and the signal/timer approval race — are concise enough to specify in pseudocode. The dual-provider evaluation harness allows direct comparison of closed and open-weight stacks on identical input, providing the empirical core of the accompanying thesis.

References

- [1] J. Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd ed., 2009.
- [2] Temporal Technologies. Workflow determinism and replay. <https://docs.temporal.io>, 2024.
- [3] The PostgreSQL Global Development Group. Row Security Policies, *PostgreSQL 16 Manual*, 2024.

- [4] A. Sharma, E. Kiciman. DoWhy: An end-to-end library for causal inference. *arXiv:2011.04216*, 2020.
- [5] S. G. Hart, L. E. Staveland. Development of NASA-TLX. *Advances in Psychology*, 1988.